

Natív Programozás Házi feladat

SZTE Szoftverfejlesztés Tanszék

2025. őszi félév

Technikai ismertető

- A programot C++ nyelven kell megírni.
- A megoldást a *Bíró* fogja kiértékelni.
 - A Feladat beadása felületen a Feltöltés gomb megnyomása után ki kell várni, amíg lefut a kiértékelés. **Kiértékelés közben nem szabad az oldalt frissíteni vagy a Feltöltés gombot újból megnyomni** különben feltöltési lehetőség veszik el!
- Feltöltés után a *Bíró* a programot g++ fordítóval és a
-std=c++20 -Wall -Werror -static -O2 -DTEST_BIRO=1
paraméterezéssel fordítja és különböző tesztesetekre futtatja.
- A program működése akkor helyes, ha a tesztesetek futása nem tart tovább 5 másodpercnél és hiba nélkül (0 hibakóddal) fejeződik be, valamint a program működése a feladatkiírásnak megfelelő.
- A *Bíró* által a `riport.txt`-ben visszaadott lehetséges hibakódok:
 - Futási hiba 6: Memória- vagy időkorlát túllépés.
 - Futási hiba 8: Lebegőpontos hiba, például nullával való osztás.
 - Futási hiba 11: Memória-hozzáférési probléma, pl. tömb-túlindexelés, null pointer használat.
- A beadások számát és a beadási határidőt a bíróban lehet megtekinteni.
- A programban a `#ifndef TEST_BIRO` és a hozzá tartozó `#endif` közötti rész nem lesz figyelembe véve a kiértékelés során.
- A bíróba egyetlen .cpp forrásfájl lehet feltölteni (zip fájl nem fogja kiértékelni).
- Technikai hiba esetén a technikai hiba elhárulása után a hallgató folytathatja a ZH-t, és a ZH határidejét a hibának megfelelően módosítjuk. Ha a hallgató a technikai hiba miatt nem képes folytatni a ZH-t, akkor az adott alkalom igazolt hiányzásnak minősül.
- A ZH megírásához hivatalos fényképes igazolvány szükséges. Ennek hiányában a ZH nem teljesíthető.

Általános követelmények, tudnivalók

- Csak a leírásban szereplő osztályokat, metódusokat és adattagokat kell megvalósítani, egyéb dolgokért nem jár plusz pont.
- Minden metódus, amelyik nem változtatja meg az objektumot, legyen konstans! Ha a paramétert nem változtatja a metódus, akkor a paraméter legyen konstans!
- string összehasonlításoknál az egyezés a pontos egyezést jelenti, azaz ha kis-nagy betűben térnek el, akkor már nem tekinthetők egyenlőnek (pl. a "piros" != "Piros")
- A leírásokban bemutatott példákban a stringek köré rakott idézőjelek nem részei az elvárt kimenetnek, azok csak a string határait jelölik. Például ha az szerepel, hogy a példa bemenetre az elvárt kimenet az, hogy "3 alma", akkor az elvárt kimenet idézőjelek nélkül az 3 alma, de a szóköz szükséges!
 - A tesztesetekben nem lesz ékezetes szöveg kiírása.
- Az elvárt kimeneteknek karakterről karakterre olyan formátumúnak kell lennie, ami a feladatban le van írva (szóközöket és sortöréseket is beleértve).
- Ha az objektum másolása nem triviális (azaz a fordító által generált másolás nem elegendő), akkor a megfelelő másolást is meg kell valósítani.

Kiindulási projekt, megoldás feltöltése

- **A megoldáshoz az előre kiadott osztályok módosítása szükséges lehet.**
 - Nem minden ZH esetében van kiindulási projekt.
- Feltöltéskor ezeket az osztályokat is fel kell tölteni és a módosításokat is pontosíthatja a bíró!
- Egyes tesztesetekben a bíró módosított osztályt is használhat ezen kiinduló osztályok helyett, ezzel tesztelve a valóban helyes működést!

Zh alatt használható segédanyag

- A ZH során használható segédanyag elérhető bíróban.
 - <https://biro.inf.u-szeged.hu/kozos/native/>

Dinamikus memória követése

Ebben a feladatban lehetőség van egy speciális konstrukció használatára ami segít az allokált memória követésére. Amennyiben a `new` után egy `TRACK_INFO`-t írunk a `biro` rendszere ki tudja írni, hogy hol került allokálásra az adott memória terület ami nem lett felszabadítva (bizonyos tesztek esetén). Ezt az információt a `testeset.stdout` kiterjesztésű fájljába írja bele.

Példa használat: `new TRACK_INFO Fajl(...)`

Példa kimenet az `stdout` fájlban:

```
[ FAIL ] Memory error occured. Here is the map of the allocated memory addresses:
---| FAIL | 0xd7ee40 is not freed. filename: feladat.cpp:123 in function: <function>
---| FAIL | 0xd7eeb0 is not freed. filename: feladat.cpp:123 in function: <function>
---| OK   | 0xd7ef20 is nicely freed.
---| OK   | 0xd7ef50 is nicely freed. filename: feladat.cpp:133 in function: <function>
```

Figyelem! Amennyiben nem `biro-n` kerül fordításra a program, ilyen memória allokáció követés nem történik.

Bevezető

A feladat a „fájlrendszer megvalósítása”, amit azért számos egyszerűsítést tartalmaz a valós fájlrendszerhez képest. A főbb feladatok (részletes feladatlista lejjebb):

- `Fajl` és `Könyvtar` osztályok megvalósítás, amelyek a közös `FRElem` osztályból származnak.
- A könyvtár képes legyen további fájlokat és könyvtárakat eltárolni, és név alapján különböztesse meg azokat.
- Képesen legyünk navigálni a könyvtárakban.
- A könyvtárat lehessen rekurzívak kilistázni.
- Egy könyvtárat képesek legyünk rekurzívan lemásolni vagy törölni.

Kiindulás

`FRElem` osztály

Az `FRElem` egy olyan absztrakt osztály, amelyik a fájlnak és a könyvtárnak a közös őse.

- Minden elemnek van neve, és mindegyik tudja, hogy ki a szülője.
 - Ezeket példányosításkor meg kell adni ebben a sorrendben.
 - * Szülő csak `Könyvtar` típusú lehet.
 - Mindkettőhöz van getter (`getNev`, `getSzulo`).
- A `getTeljesUtvonal` metódus visszaadja az adott elem teljes útvonalát `"/"` karakterrel elválasztva. Példák:
 - Gyökér könyvtár esetén: `"/"`

- Egyéb példák: `"/home/user/prog2/feladat.cpp"` vagy `"/usr/bin"`

A kiindulási feladatban kapott osztályok módosítása vagy bővítése szükséges!

Feladatok

Fajl osztály

A Fajl osztály a fájlrendszeren található fájlokat reprezentálja.

- A Fajl osztály az FRElem osztályból öröklődik.
- Létrehozáskor a nevét (`string`) és a szülőt (`Konyvtar*`) meg kell adni.
 - A fájl „nem lóghat a levegőben”, csak úgy lehet létrehozni, hogy egyből egy könyvtárban van (lásd `Konyvtar::operator%`).
- A fájlnek lehet tartalma, ami csak szöveg lehet.
 - Ehhez legyen setter (`setTartalom`) és getter (`getTartalom`).
- Lehesse törölni a fájl az `rm` metódus segítségével, és ekkor a fájl végleg megszűnik.

Konyvtar osztály

A fájlrendszer könyvtárát reprezentálja, ami fájlokat és könyvtárakat tartalmazhat, de **azonos nevűből csak egyet**. Előfordulhat, hogy ehhez kellhet egy összehasonlítást végző struktúra/operátor implementáció (lásd "Segítség" alfejezet).

- A fájlrendszer rendelkezik egy gyökérrel, aminek a neve `"/`.
 - Ezt nem a felhasználónak kell létrehoznia, a program indulásakor létezik.
 - A `getGyokerPtr` **statikus** adja vissza a pointert a gyöker könyvtárra.
 - A `getGyoker` **statikus** metódus adjon vissza egy referenciát a gyöker elemre.
 - Olyan megoldást válassz, ami nem okoz memóriaszivárgást akkor sem, ha a gyöker elem a program befejezésekor is létezik.
 - * Vagy meg kell oldani a felszabadítását a programból való kilépéskor.
- Könyvtárat csak egy létező könyvtár gyerekeként lehet létrehozni a `/` operátorral.
 - Az operátor paramétere az új könyvtár neve.
 - Hozzunk létre új könyvtárat a paraméterben kapott névvel, és kössük be az aktuális könyvtár alá.
 - * A visszatérési értéke az új könyvtár legyen (`Konyvtar&`), így az operátor láncba fűzhető.
 - * Például, a `Konyvtar::getGyoker() / "elso" / "masodik"`; utasítás hatására a gyöker könyvtár alatt létrejön egy "elso" nevű könyvtár, ami alatt meg egy "masodik".

- Ha már volt ilyen nevű elem a könyvtárban, akkor dobjon `std::exception-t`.
- Fájl a `%` operátorral lehet létrehozni egy könyvtárban.
 - Paraméterben megkapja a fájl nevét és létrehozza az adott könyvtárban.
 - * Például, a `Konyvtar::getGyoker() / "feladat.cpp"`; utasítás hatására a gyökér könyvtár alatt létrejön egy `"feladat.cpp"` nevű fájl.
 - * A fájl tartalma üres string.
 - Az operátor a létrehozott fájlt adja vissza (`Fajl&`).
 - Ha már volt ilyen nevű elem a könyvtárban, akkor dobjon `std::exception-t`.
- Könyvtárváltásra a `>>` operátort biztosít lehetőséget, aminek a paramétere egy string.
 - Visszatérési típusa `Konyvtar&` legyen.
 - Ha a paraméterben kapott string `".."`, akkor az a szülő könyvtár felé történő lépést jelenti.
 - * Ha a gyökerkönyvtáron állunk, akkor önmagát adja vissza.
 - * Ha nem a gyökér könyvtáron állunk, akkor a szülő könyvtárat adja vissza.
 - Ha a paraméter nem `".."`, akkor abba a könyvtárba akarunk lépni¹.
 - * Ha van ilyen könyvtár, akkor azt adja vissza.
 - * Különben dobjon `std::exception` kivételt.
- Az `ls` metódus segítségével ki lehessen listázni az elemeket **rekurzívan**.
 - Az első paramétere egy `std::ostream` amire írja ki az elemeket.
 - Az elemeket ábécé sorrendben írja ki.
 - A gyerek elemeket a szülőhöz képest két szinttel beljebb húzva jelenítse meg.
 - Nincs visszatérési értéke.
 - A fájlok mögé zárójelben írja oda, hogy `"fajl"`.

Példa:

```

/
  első
  harmadik
    alkonyvtar
      readme.txt (fajl)
      feladat.cpp (fajl)
      tovabbi_konyvtar
  masodik
    1
    11
    2
  negyedik
  otodik

```

¹Nem foglalkozunk olyan speciális esetekkel, mint a `"."` vagy `"/"`.

- A `cp` segítségével lehessen rekurzívan másolni, ahol paraméterben kapott elemet (fájlt vagy könyvtárat) kell az adott könyvtárba másolni.
 - Ha az adott könyvtárban már van olyan nevű fájl vagy könyvtár, amit ide szeretnénk másolni, akkor dobjon `std::exception` kivételt.
 - Ha nincs ilyen nevű elem, akkor **másolja** ide az adott elemet.
 - * Ha fájl, akkor a fájl a tartalmával együtt le kell másolni.
 - * Ha könyvtárat kell másolni, akkor az adott könyvtárat az összes alatta található könyvtárral és fájlal együtt rekurzívan másolja le.
 - * Nem kell azzal foglalkozni, hogy a `cp` egy olyan könyvtárat kap, ami az adott könyvtárnak az őse, és így végtelen lenne a másolás.
 - Nincs visszatérési értéke.
- Lehessen fájlt vagy könyvtárat is törölni az `rm` metódus segítségével.
 - Könyvtár törlése esetében a könyvtárhoz tartozó összes fájl és alkönyvtárat törölni kell, majd magát a könyvtárat is.
 - **A gyökér könyvtárat soha sem töröljük.**
 - * Ha a gyökér könyvtárra alkalmazzuk, akkor az összes alkönyvtárt törölni kell, de magát a gyökér könyvtárat nem szabad törölni.
 - Nincs visszatérési értéke.

Segítség

Amennyiben, egy `std::set` pointereket tárolunk alapértelmezetten minden elem egyedi lesz hiszen a memóriacímek egyediek. Annak érdekében, hogy ne memóriacím szerint történjen az elemek vizsgálata egy extra összehasonlító operátort kell megadnunk a halmaznak mint template paraméter.

Vegyük példának az alábbi kódrészletet:

```
class osszehasonlitas {
public:
    bool operator()(const FRElem* f1, const FRElem* f2) const {
        return f1->getNev() < f2->getNev();
    }
};

std::set<FRElem*, osszehasonlitas> data;
```

Ekkor a `data` halmazba való új elem illesztésekor az `osszehasonlitas` eljárás hívás operátora fut le, ezzel vizsgálva az egyediséget.